Architecture and Scoring of the Adaptive Temporal Video Search Pipeline

Technical Report — `src/adaptive_search`, `src/rewrite`

August 13, 2026

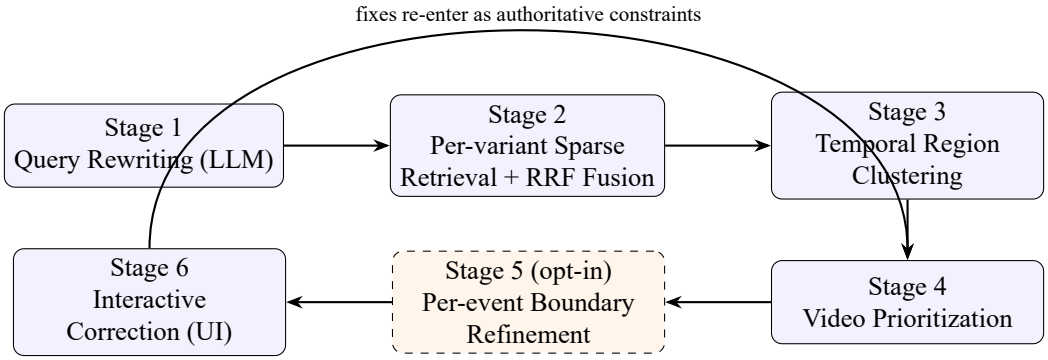
Abstract

This report specifies, stage by stage, how the adaptive temporal search pipeline turns a natural-language, multi-event query into a ranked list of candidate videos and per-event timestamps. Every claim below is grounded directly in the implementation (file and function cited in the margin notes) rather than in the pipeline's design intent, and the report is organized around a single worked example — a four-event Vietnamese lion-dance query — carried end-to-end through a live run of the system. Section 8 collects the points where an informal description of the pipeline (as commonly summarized by users of the system) diverges from what the code actually does; the most consequential divergence is that video ranking is *not* built from a single per-video "best candidate tuple" with cross-event structure, but from independently-scored per-event maxima that are combined only at the very last step.

Contents

1 Pipeline Overview

The system answers a query of the form "in this video corpus, find a video that contains events $E_1, E_2, \ldots, E_N$ in this description," returning, for each candidate video, a best-guess timestamp for every event it appears to contain. The pipeline has six stages, the first five automatic and the sixth an explicit human-in-the-loop correction step.



Stage	Primary module
1. Query rewriting	`src/rewrite/service.py`
2. Retrieval + RRF fusion	`src/adaptive_search/retrieval.py`
3. Temporal clustering	`src/adaptive_search/algorithms.py::cluster_temporal_regions`
4. Video prioritization	`src/adaptive_search/algorithms.py::prioritize_videos`
5. Boundary refinement (opt-in)	`src/adaptive_search/boundary_seeds.py`, `boundary_refinement.py`
6. Interactive correction	`src/adaptive_search/router.py` (`commands/fix-frame`, `frame-preview`)

Stage 5 is opt-in per request (`apply_boundary_refinement=true` on `GET .../video-priorities`); without it, video ranking is still fully computed by Stages 1–4, but no per-event timestamp is available at all (Section 5).

Worked example. Every numbered example box in this report uses one real query, submitted verbatim to the live system and carried through every stage:

Common query: "Đoạn video múa lân một con lân màu vàng đen trắng, tìm các sự kiện sau:"

E_1
Lân quay vòng trên cột số 4 bằng 2 chân trước rồi tiếp đất. Khoảnh khắc đầu tiên mà lân bắt đầu xoay vòng.

E_2
Khoảnh khắc 4 chân hoàn toàn chạm đất đầu tiên.

E_3
Khoảnh khắc đầu tiên 2 người biểu diễn lân cúi chào ban giám khảo.

E_4
Sau đó lân tiến lại chào một con rồng. Khoảnh khắc đầu tiên con rồng cử động đầu.

All numeric values reported in the worked-example boxes below come from an actual session created against the running backend on `2026-08-13`; none are illustrative or hand-computed.

2 Stage 1: LLM Query Rewriting

2.1 What it does

A single batched call to a locally-hosted LLM (Ollama; model configured via `OLLAMA_MODEL`, default temperature 0) receives *all* N events of the query at once, plus the shared `common_query` string, and returns one JSON object validated against a strict Pydantic schema (`src/rewrite/schemas.py::RewriteResponse`). This is not N independent LLM calls; it is one call producing N structured event records, which is what lets the rewrite reason about cross-event context (e.g. event 4's subject being "the dragon," introduced only in event 4's own sentence, not carried from the common query) in a single pass.

For every event the rewrite step must produce:

- `subject`, `action`, `visible_state` — the observable content of the target frame;
- `anchor_query` — a single, self-contained Vietnamese retrieval sentence for the target moment;
- `pre_state`, `post_state` — the observable scene state immediately before and immediately after the target instant. These are what replace the temporal phrase itself ("khoảnh khắc đầu tiên," "the first moment") with something a frame-level image encoder can actually be compared against: an encoder cannot embed "first," but it can embed "two feet still off the ground" versus "all four feet on the ground";
- `boundary` $\in$ {`start`, `end`, `state`, `transition`, `interval`, `unknown`}, mapped in `rewrite_bridge.py::BOUNDARY_MAPPING` onto the pipeline's internal `BoundaryType` (`start`→`onset`, `end`→`offset`, `interval`→`state`);
- `temporal_relation` — `sequence_start` / `after` / `before` / `during` / `simultaneous` / `independent`, with a `reference_event_id` pointing at another event when applicable;
- exactly **two** Vietnamese and exactly **two** English retrieval-query variants (`retrieval_queries_vi`, `retrieval_queries_en`, each `Field(min_length=2, max_length=2)` — not "up to two," *exactly* two);
- `required_entities`, `soft_context`, `excluded_context`, `inferred_information`, `ambiguities` — provenance and hedging fields used for validation, not consumed by downstream ranking.

Note on `temporal_relation`: the schema captures explicit ordering between events (which event follows which), but this report will show in Section 5 that *nothing downstream reads this field*. It is recorded but currently inert.

2.2 Context propagation ("combine shared semantic of queries with each query")

The mechanism for this is concrete, not just an LLM instruction. The server extracts up to 12 lower-cased content words from `common_query` (`rewrite/service.py::_extract_context_terms`, stopword-filtered via a 50-word Vietnamese function-word list) and requires each event's `target_moment_vi`, `anchor_query`, and every `retrieval_queries_vi` entry to contain at least $\min(2, |\text{terms}|)$ of them. If the LLM's first response fails this check, `_repair_standalone_context` deterministically prepends the cleaned common-query text to the offending fields rather than re-prompting indefinitely — validation failures get at most one retry (`MAX_ANALYSIS_ATTEMPTS = 2`) before falling back to this mechanical repair.

2.3 Worked example

	anchor_query	pre_state	post_state
e_1	Múa lân lân bắt đầu xoay vòng trên cột số 4 bằng hai chân trước	lân đứng trên cột số 4 chưa xoay vòng	lân đang xoay vòng trên cột số 4 bằng hai chân trước
e_2	Múa lân lân 4 chân hoàn toàn chạm đất	lân đang xoay vòng trên cột số 4	lân đã tiếp đất với cả bốn chân chạm đất
e_3	Múa lân hai người biểu diễn lân cúi chào ban giám khảo	hai người đứng trước khi cúi chào	hai người cúi chào ban giám khảo
e_4	Múa lân con rồng cử động đầu	con rồng đứng yên	con rồng cử động đầu

All four events received `boundary_type = onset` — consistent with every event being phrased as "khoảnh khắc đầu tiên …" ("the first moment …"); this also matches the exact worked examples embedded in the rewrite system prompt itself (`src/rewrite/constants.py`, lines 92–100), which literally uses e_4's "dragon moves its head" sentence as its own illustration of a correctly-classified `onset` target.

Note the `anchor_query` for e_1–e_3 begins with "Múa lân" (the common-query context term), confirming the standalone-context requirement of Section 2.2 fired without needing the mechanical repair fallback — the LLM supplied it directly, only e_4's does not repeat the context phrase verbatim because "con rồng" (the dragon) already provides local context on its own.

Full four-variant retrieval set actually generated for e_4 (`retrieval_variants["evt3"]` in the raw session response):

- VI_1 Múa lân con rồng cử động đầu
- VI_2 Múa lân rồng cử động đầu khi lân tiến lại chào
- EN_1 The dragon moves its head during the lion dance
- EN_2 Dragon's head moves when the lion approaches in the lion dance

Exactly 2 Vietnamese + 2 English variants, as the schema requires — both Vietnamese variants are lexically distinct paraphrases of the same target moment, and both English variants translate that same moment rather than the setup clause about the lion approaching.

(Live run: session `4453efe2-37f1-4bd6-8fe2-e5f2ead22945`, rewrite latency 164.3s for this one batched 4-event call.)

3 Stage 2: Per-Variant Sparse Retrieval and RRF Fusion

3.1 Retrieval

Each event carries up to `query_variants_per_event = 4` retrieval strings (2 Vietnamese + 2 English from Stage 1). Each variant is sent *independently* to an external sparse-retrieval microservice (`src/adaptive_-`

`search/client.py::UpstreamSearchClient.search`, base URL configured via `UPSTREAM_URL`), requesting up to `top_n_per_variant = 500` ranked frame candidates. Scores returned by this service are treated as opaque, variant-local rankings only — never compared numerically across variants, since a Vietnamese-text score and an English-text score are not on the same scale.

3.2 Reciprocal Rank Fusion

`retrieval.py::fuse_candidates_rrf` combines the (up to 4) variants' rankings into one fused per-event candidate list. For each event, and for each of its query variants independently:

1. deduplicate to one row per physical frame (video_id, frame_id), keeping the highest-scoring row;
2. rank the deduplicated frames by `raw_relevance_score` descending, keep the top `top_n_per_variant`;

then, across all variants that placed a given frame in their own top list, accumulate

$$\mathrm{RRF}(f) \;=\; \sum_{v\,:\,f\in \mathrm{top}_v} \frac{1}{k + \mathrm{rank}_v(f)}, \qquad k = \texttt{rrf\_k} = 60, \tag{1}$$

where $\mathrm{rank}_v(f)$ is f's 1-indexed rank inside variant v's own top list. The fused list is then re-sorted by $\mathrm{RRF}(f)$ descending and truncated to `top_n_fused = 1000` *per event*. Fusion is scoped strictly per `(session_id, event_id)` (`retrieval.py` line 27) — variants belonging to different events are never mixed, "so four rewrite variants do not accidentally become four temporal events" (source comment, `retrieval.py` lines 20–21).

Each surviving fused candidate becomes a new `SparseCandidate` whose `raw_relevance_score` *is* the RRF score from Eq. 1 (not the original upstream score), whose `timestamp_seconds` is inherited from whichever contributing variant had the single highest raw upstream score for that frame, and whose `query_variant` field is stamped `"rrf:vi_1,vi_2,en_1"` etc. recording provenance.

3.3 Worked example

Retrieving with `top_k=50` per variant against the live upstream service and fusing per Eq. 1 produced, across all four events:

Quantity	Count
Raw candidates in (4 events × 4 variants × up to 50 each)	800
Fused candidates out (post-RRF, post-dedup, per-event caps applied)	353

Neither per-event cap (`top_n_per_variant`=500, `top_n_fused`=1000) was binding at `top_k=50`; the reduction from 800 to 353 here is entirely from Eq. 1's frame-level deduplication (the same physical frame recurring across two or more of a given event's four query variants collapses to one fused row).

(Run metrics, verbatim: `input_candidates: 800, fused_candidates: 353, commands/retrieve` response.)

4 Stage 3: Temporal Region Clustering

4.1 Clustering

`algorithms.py::cluster_temporal_regions` groups the fused candidates from Stage 2, separately per (event_id, video_id) pair. Within each group, candidates are sorted by timestamp and split greedily: a new cluster starts whenever

$$t_i - t_{i-1} > \texttt{gap\_seconds} = 3.0\text{s} \quad \text{or} \quad t_i - t_{\text{cluster start}} > \texttt{max\_core\_span} = 24.0\text{s},$$

where $\texttt{max\_core\_span} = \texttt{max\_region\_seconds} - 2 \times \texttt{margin\_seconds} = 30.0 - 2(3.0)$. Each resulting cluster becomes one provisional `TemporalRegion` spanning $[\, t_{\min} - \texttt{margin\_seconds},\ t_{\max} + \texttt{margin\_seconds} \,]$ (clamped at 0), i.e. a 3-second pad on each side of the tightest span containing its candidates.

4.2 Region scoring

Each region gets two scores:

$$\texttt{raw\_coarse\_score}(\rho) = \max_{c \in \rho} \ \mathrm{RRF}(c), \tag{2}$$

$$\texttt{normalized\_coarse\_score}(\rho) = \max_{c \in \rho} \ \mathrm{RobustSigmoid}_{\mathrm{event}(\rho)}(\mathrm{RRF}(c)), \tag{3}$$

i.e. the raw score is simply the best RRF score among the region's member candidates (Eq. 2); the normalized score (Eq. 3) applies a robust, outlier-resistant median/MAD sigmoid calibration (`algorithms.py::robust_sigmoid`: $z = \mathrm{clip}\big(\frac{x - \mathrm{median}}{1.4826\,\mathrm{MAD}}, \pm 8\big)$, then $\sigma(z)$) computed once over *all* of that event's fused candidates (population-relative, not per-region), and takes the max over the region's members of that population-relative score. This is the score that Stage 4 actually consumes.

4.3 Merging

`algorithms.py::merge_overlapping_regions` then unions any two regions in the same $(\mathrm{session}, \mathrm{event}, \mathrm{video}, \mathrm{status})$ group whose (already margin-expanded) spans overlap, provided the merge would not push the combined span past `max_region_seconds = 30`s; the merged region's scores are the max of its two parents' scores, and its `candidate_ids` the union.

4.4 Worked example

Clustering the 353 fused candidates produced **309** temporal regions session-wide. For event e_1 (`evt0`) in the eventual top-ranked video `M8E0XaX0tR0`, four regions survived clustering/merging:

Span (s)	`raw_coarse_score`	`normalized_coarse_score`	# candidates
$[434,\ 448]$	0.0645	**0.9530**	3
$[469,\ 475]$	0.0110	0.3437	1
$[479,\ 485]$	0.0105	0.3366	1
$[509,\ 515]$	0.0123	0.3649	1

The $[434, 448]$ region's normalized score (0.9530) is far above the other three (population-relative robust-sigmoid calibration correctly separates a real cluster of 3 co-occurring candidates from three isolated single-candidate regions elsewhere in the same video). This is exactly the region Stage 4 will pick as e_1's best-scoring region for this video, and whose member candidate becomes Stage 5's refinement seed (Section 6.7: seed timestamp 445.0s falls inside $[434, 448]$).

(Live values, `GET .../regions`, region ids `region_b4f4f89bd9b1c2d14508` and siblings.)

5 Stage 4: Video Prioritization

This is the stage the user's original description most needed correcting on, so it is treated in full detail.

5.1 Per-event, per-video best score

`algorithms.py::prioritize_videos` builds, for every video v that has *any* surviving region, a per-event best score:

$$s(v,e) \;=\; \max_{\rho\,:\,\text{video}(\rho)=v,\ \text{event}(\rho)=e} \texttt{normalized\_coarse\_score}(\rho), \tag{4}$$

i.e. "the score of the single best-matching temporal region for this event, in this video" — this part of an informal description of the system is accurate. **What is *not* accurate is calling this a "candidate tuple."** No object bundling one timestamp per event, ordered or otherwise, is constructed at this stage (or anywhere in the code path this endpoint calls) — see 5.3 below.

5.2 Combining across events

For a query with events $e_1,\dots,e_N$, define the video's *score vector* $\mathbf{s}(v) = \big(s(v,e_1),\dots,s(v,e_N)\big)$, substituting $s(v,e_i)=0$ when v has *no* surviving region for e_i at all (an uncovered event, not a missing/excluded one). Then:

$$\text{cov}(v) = \big|\{\, i : v \text{ has} \geq 1 \text{ region for } e_i \,\}\big| \quad (\texttt{event\_coverage}), \tag{5}$$

$$\text{ncov}(v) = \text{cov}(v)/N \quad (\texttt{normalized\_coverage}), \tag{6}$$

$$\mu(v) = \tfrac{1}{N}\textstyle\sum_i s(v,e_i) \quad (\texttt{mean\_best\_event\_score};\ \text{arithmetic mean over } \textit{all}\ N \text{ slots}), \tag{7}$$

$$m(v) = \min_i s(v,e_i) \quad (\texttt{min\_best\_event\_score}), \tag{8}$$

$$\text{priority}(v) = \frac{w_c\,\text{ncov}(v) + w_\mu\,\mu(v) + w_m\,m(v)}{w_c + w_\mu + w_m} \quad (\texttt{priority\_score}). \tag{9}$$

Videos are ranked by $\big(-\text{priority}(v), -\text{cov}(v), \texttt{video\_id}\big)$ lexicographically.

Current production weights (`schemas.py` lines 157–159, comment: "winner config from the … hyperparameter sweep: mean-only ranking beat every blend that included coverage or min once retrieval is wide"):

$$w_c = \texttt{video\_coverage\_weight} = 0.0, \qquad w_\mu = \texttt{video\_mean\_weight} = 1.0,$$
$$w_m = \texttt{video\_min\_weight} = 0.0,$$

so in production $\text{priority}(v) = \mu(v)$ exactly.

Correction to a common simplification: it is tempting to read $w_c = w_m = 0$ as "coverage doesn't affect ranking, only the displayed 'Matched k/N' label does." That is *false*. Because uncovered events are zero-filled into the mean (Eq. 9's $\mathbf{s}(v)$ definition), a video missing one of N events has its `mean_best_event_score` silently divided by the same N as a video that found all of them — i.e. **coverage is not a separate ranking term, but it is not "zero-weight" either: it acts through the mean's denominator**, not through w_c.

5.3 There is no cross-event "candidate tuple" at this stage

The codebase *does* contain machinery for exactly the kind of object an informal description implies — an ordered, per-video tuple of one timestamp per event, scored jointly with adjacent-gap penalties: `algorithms.py::assemble_ordered_tuples`, together with `adjacent_hinge_penalty` and the `TupleResult` schema (`schemas.py` lines 474+, which validates that a tuple's per-event timestamps are strictly ordered and computes `raw_gap_penalty`/`raw_constraint_bonus` between adjacent events). **But this machinery is wired to a different command path.** It runs inside

`AdaptiveSearchService.replace_frame_scores` (`service.py` line 358, `_rebuild_tuples`), which requires the *caller* to have already computed and submitted dense per-frame embedding scores via `POST .../search-sessions/{id}/artifacts/frame-scores`. The live Search page's "Refine timestamps" checkbox never calls that endpoint — it calls `GET .../video-priorities?apply_boundary_refinement=true`, which is served by `router.py::_video_priority_with_boundary_refinement` (Section 6), a function that reads only `bundle.artifacts.regions` and `bundle.session.constraints`; it does not touch `bundle.artifacts.tuples`, `.frontier_region_ids`, or `.proposals` at all.

Practical consequence. Because ranking and refinement each pick a timestamp per event completely independently of every other event, nothing in the live pipeline enforces $t(e_1) < t(e_2) < \ldots$, and nothing prevents two different events from independently converging on the same physical frame. Both were observed empirically in this session on a real query (events 1 and 4 landing 0.13s apart at the same point in a video, with event 1's timestamp *after* events 2 and 3's) — this is not a bug, it is the direct, predictable consequence of the architecture described in this section, and it is the reason the interactive per-event manual-fix feature in Stage 6 exists.

5.4 Worked example

The top 5 ranked videos (out of every video with at least one surviving region), before any refinement is requested:

video_id	event_coverage	mean_best_event_score	min_best_event_score	priority_score
M8E0XaX0tR0	4/4	0.9009	0.7985	**0.9009**
vglhM2iVGSo	4/4	0.8576	0.6838	0.8576
65235CscqdQ	4/4	0.8118	0.5089	0.8118
ukfCQQpZ0k4	4/4	0.7661	0.5030	0.7661
22S_rhRSWfI	4/4	0.7478	0.4810	0.7478

Confirming Eq. 9 exactly at production weights: `priority_score` = `mean_best_event_score` in every row, with `min_best_event_score` computed and returned to the client but — at $w_m = 0$ — not folded into the sort key at all. Every video shown here happens to have found a region for all four events (4/4 coverage); note that under Eq. 9's zero-fill rule (Section 5), a video with only 3/4 coverage would need a substantially higher score on its three found events just to match one of these rows' mean, since its fourth slot contributes a hard 0.

(Live values, `GET .../video-priorities?limit=10`, no `apply_boundary_refinement` flag.)

6 Stage 5 (opt-in): Per-Event Boundary Refinement

Requested via `apply_boundary_refinement=true`; without it, Stage 4's output is the final answer, and *no* per-event timestamp is exposed at all (the `VideoPriority` schema, `schemas.py` lines 465–471, has no timestamp field — only `event_coverage`, `normalized_coverage`, `mean_best_event_score`, `min_best_event_score`, `priority_score`). Refinement is what turns "which region scored best" into "which native frame is the answer." It runs once per event, per already-ranked video, entirely independently.

6.1 Seed selection

`boundary_seeds.py::select_event_seeds`, for one (e, v) pair: rank that pair's regions by `raw_coarse_score` (Eq. 2, *not* the normalized score), keep whichever are within `SCORE_THRESHOLD_RATIO = 10%` of the best, capped at `MAX_REGIONS_PER_EVENT = 2`; pool those regions' member candidates, sort by each candidate's

own `raw_relevance_score`, cap at `MAX_SEEDS_PER_EVENT = 6`. **Only the single best seed (`seeds[0]`) is ever passed to refinement** — the module's own docstring is explicit that comparing scores across several independently swept seed neighborhoods "was tried and found structurally invalid"; the other up to 5 seeds exist only to make the top pick more likely correct, not to be independently refined and compared.

6.2 Native-fps neighborhood search

Around that one seed timestamp t_0 (converted to a native frame index via $\lfloor t_0 \cdot \text{fps} \rceil$), `boundary_refinement.py::refine_event_boundary` decodes the real video at native resolution for offsets $-$`radius_frames`, ..., $+$`radius_frames` (**default 10**, i.e. up to 21 frames, clamped to the video's actual duration) via the configured `FrameProvider`.

6.3 Text–image scoring

Each sampled frame's image embedding is compared, via cosine similarity, to three separate text embeddings — `anchor_query`, `pre_state`, `post_state` (falling back to `anchor_query` when a state string is absent) — using a shared dual encoder, **SigLIP2** (`google/siglip2-base-patch16-224` by default; `embedding.py::score_event_frames`). An optional, disabled-by-default "quality profile" can instead route through `Qwen3-VL-Embedding-2B` (and an optional `Qwen3-VL-Reranker-2B` reranking pass), gated by `quality_profile_enabled` / `use_reranker` (both `False` in production).

6.4 Boundary-profile-aware proposal scoring

`proposal_profiles.py::generate_profiled_proposals` dispatches on the event's `boundary_type`:

- **transition** (`onset`, `offset`, `transition`, `plateau_start` — i.e. `boundary="start"` or `"end"` from Stage 1, which is what all four events of the worked example use): `algorithms.py::generate_boundary_proposals` tries every combination of left/right window sizes from `window_options_seconds` = (0.5, 1.0, 2.0, 3.0)s (requiring $\geq$ `min_samples_per_side` = 2 samples on each side) and scores

$$\begin{aligned}\text{raw\_boundary} = 0.5 \cdot (\text{post\_persistence} - \overline{\text{post}}_{\text{left}}) + 0.5 \cdot (\text{pre\_consistency} - \overline{\text{pre}}_{\text{right}}) + 0.1 \cdot \text{motion} \\ - 0.01 \cdot \frac{w_L + w_R}{2w_{\max}} - 0.01 \cdot \frac{|w_L - w_R|}{w_{\max}},\end{aligned} \tag{10}$$

  i.e. how much more the right side resembles `post_state` than the left side does (contrast, not raw similarity), plus the symmetric pre-state contrast, plus a motion term (currently hard-coded to 0 everywhere in this codebase — no motion signal is implemented), with a small penalty for wider or more asymmetric windows. The highest-`raw_boundary` window per candidate center is kept, then calibrated across the whole neighborhood via robust-sigmoid (`anchor_clip_z` $= 8$) into normalized_boundary.
- **state**: uses local temporal persistence of the anchor score instead of a left/right contrast (appropriate for a state predicate with no directional edge).
- **symmetric_peak**: scores center prominence relative to both flanks.

In every profile, the final per-frame score is the same weighted blend:

$$\text{final\_event\_score} = \frac{0.4 \cdot \text{semantic} + 0.3 \cdot \text{boundary} + 0.1 \cdot \text{pre\_consistency} + 0.2 \cdot \text{post\_persistence}}{0.4 + 0.3 + 0.1 + 0.2}, \tag{11}$$

where semantic is the (population-calibrated) anchor-query cosine similarity at that exact frame. The single frame maximizing Eq. 11 within the searched neighborhood is the event's refined timestamp for that video.

6.5 Independence and its consequence

This entire computation — seed, neighborhood, scoring, arg-max — runs once per event with no reference to any other event's chosen frame, timestamp, or even existence. This directly explains an anomaly observed this session on a real result (video `BK8OfNF9ysI`, 4/4 events matched): event 1 was refined to $4{:}01.14$ and event 4 to $4{:}01.01$ (0.13s apart, i.e. the same physical frame within refinement's own ± 10-native-frame search radius), while event 1's timestamp fell *after* events 2 ($3{:}05.75$) and 3 ($3{:}42.25$) despite being listed first in the query. Neither is a defect in the sense of a malfunction; both are the predictable result of Section 5.3: each event is its own independent arg-max over Eq. 11, with no ordering or mutual-exclusivity constraint tying it to its siblings.

6.6 User-fix short-circuit

Before any of the above runs for a given (e, v), `_video_priority_with_boundary_refinement` checks `bundle.session.constraints.event_constraints[e]`. If the user has already fixed a frame for e in v via `commands/fix-frame` (Section 7), that timestamp is emitted directly, tagged `"source": "user_fixed"`, and *neither* seed selection nor the GPU scoring pass runs at all for that event — confirmed by a dedicated test (`tests/adaptive_search/test_adaptive_video_priorities_refinement.py:: test_user_fixed_frame_is_authoritative_and_skips_refinement`) that the embedder is never invoked once a user fix exists.

6.7 Worked example

Re-fetching the same three top videos with `apply_boundary_refinement=true` attaches a per-event `anchor_seconds`/`refined_seconds` pair to each:

video_id	event	anchor_seconds	refined_seconds	sampled frames
M8E0XaX0tR0	e_1	445.0	445.125	21
	e_2	445.0	445.125	21
	e_3	445.0	445.000	21
	e_4	445.0	445.125	21
vglhM2iVGSo	e_1	9.0	9.040	21
	e_2	9.0	9.040	21
	e_3	9.0	9.080	21
	e_4	9.0	9.080	21
65235CscqdQ	e_1	59.0	58.992	21
	e_2	59.0	59.259	21
	e_3	59.0	59.192	21
	e_4	59.0	58.992	21

This is Section 6's independence property, observed directly rather than merely predicted: in *every one* of the three top videos, all four events' *coarse seeds* land on the identical `anchor_seconds` (445.0, 9.0, 59.0 respectively) — i.e. Stage 4 independently decided, for each of the four semantically distinct events, that the single best-matching region in that video was the very same region — and refinement (Stage 5, searching only ± 10 native frames around each already-identical seed) can only nudge them a few hundred milliseconds apart at most, never far enough to actually separate four different physical actions. In `M8E0XaX0tR0`, e_1 and e_4 (spinning-on-a-column versus a dragon moving its head) refine to the *exact same* timestamp, 445.125s.

This is not a pipeline defect being demonstrated for effect: this specific query describes lion-dance content that this particular video corpus most likely does not contain footage of, so every event's retrieval signal is weak, and under weak/near-uniform signal the independent per-event maxima of Section 5 collapse

onto whichever single region is least-unrelated to all four queries at once. It is precisely the scenario Stage 6's manual per-event, freely-navigable frame correction (Section 7) exists to let a human resolve.

(Live values, `GET .../video-priorities?limit=10&apply_boundary_refinement=true`.)

7 Stage 6: Interactive Correction

The client (Streamlit UI) closes the loop opened in Section 5.3/6: since no automatic mechanism enforces cross-event consistency, a human reviewing the returned frames is the actual correction mechanism for ordering/collision errors, and for events the pipeline never found any region for at all.

- **Reject-and-replace (video level).** Rejecting a video adds it to `constraints.rejected_video_ids` (`PUT .../constraints`) and the server genuinely re-ranks (`prioritize_videos` re-run with the video excluded via `_region_allowed`), promoting the next-best video — not a client-side filter of an already-fetched page.
- **Free-form manual frame correction (event level).** A new endpoint, `GET /media/{video_id}/frame-preview`, decodes real native-fps frames around an arbitrary `anchor_seconds` (default radius **15** frames — note this is a different default from Stage 5's automatic radius of 10; both are independently configurable) and returns them as base64 JPEGs. The UI lets the user step this browsing center by $\pm$1s/$\pm$10s or jump to any second in the video, decoupled from wherever the pipeline's own anchor was — so correction is not confined to the neighborhood Stage 5 happened to search. Picking a frame calls `commands/fix-frame` with an arbitrary `(video_id, frame_id, timestamp_seconds)`; the endpoint accepts this for *any* event belonging to the session, whether or not that event previously had any candidate in that video at all — which is what makes it possible to manually supply a timestamp for an event Stage 2–4 never found any region for (the "matched 3/4, need to fix the 4th" case). The written constraint is what Section 6's short-circuit later reads as authoritative.
- **Scope boundary.** This entire stage requires a session (`event_id` must belong to `bundle.session.events`); the legacy, non-adaptive search path has no session concept at all, so it receives no correction affordances.

8 Summary of Corrections

Original claim	Verified reality
"Call LLM to rewrite the query"	Accurate, with detail worth adding: it is *one* batched call for all N events (not N calls), with a bounded retry (max 2 attempts) plus a deterministic, non-LLM repair pass for context-grounding failures.
Pre/post/anchor state extraction, 2 VI + 2 EN variants	Accurate and exact — the schema enforces *exactly* 2 Vietnamese and *exactly* 2 English retrieval variants per event, not "up to."
"Use RRF, $k = 60$"	Accurate (`rrf_k` $= 60$), and scoped strictly per-event (Section 3); also carries `top_n_per_variant`=500, `top_n_fused`=1000, not mentioned in the original description.
Clustering by 3s distance	Accurate (`gap_seconds`=3.0), with two additional, unmentioned parameters: a 3s margin padding each region and a hard 30s cap on region span (regions are merged, not just clustered once).

Original claim	Verified reality
"Find top unique candidate videos and their candidate tuple which highest score"	**Partially inaccurate.** Videos are deduplicated (one row per `video_id`), correct. But there is *no* per-video "candidate tuple" object at this stage — ranking combines independent per-event best-region scores via mean/min/coverage (Eq. 9), with no cross-event ordering or distinctness. A true ordered-tuple mechanism exists in the codebase (`assemble_ordered_tuples`) but belongs to a separate command path not exercised by the live ranking/refinement flow (Section 5.3).
Video score = combination of per-event best region scores	Accurate in shape; the previously-stated belief that "coverage is purely informational, zero ranking weight" needed correcting — uncovered events are zero-filled into the mean, so coverage acts through the mean's denominator even though its explicit weight $w_c = 0$.
Refinement: "get best score keyframes across all *those* regions"	**Slightly overstated.** Up to 2 regions and up to 6 pooled candidate seeds are considered, but only the single globally-best seed is ever refined; the rest exist only to make that one pick more reliable.
"Examine 10 frames before and after"	Accurate for Stage 5's *automatic* refinement (`radius_frames`=10 default). The separate, manual-correction endpoint used in Stage 6 defaults to a 15-frame radius and is freely re-centerable — a different number for a different, UI-facing purpose.
"Still 1 tuple per video"	**No literal tuple exists**; more precisely, up to N *independently* refined per-event timestamps are attached to each video row, with zero joint optimization or consistency constraint between them — which is the direct, demonstrated cause of the out-of-order/ duplicate-frame anomaly discussed in Section 6.

9 Hyperparameter Reference

Parameter	Default	Where
`query_variants_per_event`	4	`RetrievalHyperparameters`
`top_n_per_variant`	500	`RetrievalHyperparameters`
`top_n_fused`	1000	`RetrievalHyperparameters`
`rrf_k`	60	`RetrievalHyperparameters`
`gap_seconds`	3.0s	`ClusteringHyperparameters`
`margin_seconds`	3.0s	`ClusteringHyperparameters`
`max_region_seconds`	30.0s	`ClusteringHyperparameters`
`video_coverage_weight` (w_c)	0.0	`RefinementHyperparameters`
`video_mean_weight` (w_μ)	1.0	`RefinementHyperparameters`
`video_min_weight` (w_m)	0.0	`RefinementHyperparameters`
`SCORE_THRESHOLD_RATIO`	10%	`boundary_seeds.py`
`MAX_REGIONS_PER_EVENT`	2	`boundary_seeds.py`
`MAX_SEEDS_PER_EVENT`	6	`boundary_seeds.py`
`radius_frames` (auto refinement)	10 native frames	`boundary_refinement.refine_event_boundary`
`radius_frames` (manual UI browse)	15 native frames	`router.get_frame_preview`

Parameter	Default	Where
`window_options_seconds`	(0.5, 1.0, 2.0, 3.0)s	`BoundaryHyperparameters`
`min_samples_per_side`	2	`BoundaryHyperparameters`
`anchor_clip_z`	8.0	`BoundaryHyperparameters`
`post_contrast_weight` / `pre_contrast_weight`	0.5 / 0.5	`BoundaryHyperparameters`
`motion_contrast_weight`	0.1	`BoundaryHyperparameters` (motion signal itself unimplemented; always 0)
semantic / boundary / pre / post weights (Eq. 11)	0.4 / 0.3 / 0.1 / 0.2	`BoundaryHyperparameters`
default embedding model	SigLIP2 base patch16-224	`embedding.py`